第一章：排序笔记

1 插入排序

代码

```
public static void insertionSort(int[] arr) {
    if (arr == null || arr.length < 2) {
        return;
    }
    for (int i = 1; i < arr.length; i++) {
        for (int j = i - 1; j >= 0 && arr[j] > arr[j + 1]; j--) {
            swap(arr, j, j + 1);
        }
    }
}

public static void swap(int[] arr, int i, int j) {
    int tmp = arr[i];
    arr[i] = arr[j];
    arr[j] = tmp;
}
```

思想

插入排序把数组分成两部分：

- 左边部分已经排好序，
- 右边部分还没有排好序。

每一步，它从未排序部分取出一个元素，并把它插入到已排序部分中的正确位置。

例子

假设

$$arr = [5, 3, 4, 1].$$

过程如下：

$$[5, 3, 4, 1]$$

把 3 插入到已排序部分 $[5]$ 中：

$$[3, 5, 4, 1]$$

把 4 插入到已排序部分 $[3, 5]$ 中：

$$[3, 4, 5, 1]$$

把 1 插入到已排序部分 $[3, 4, 5]$ 中：

$$[1, 3, 4, 5]$$

核心思想

在第 i 步时，子数组

$$arr[0 \ldots i-1]$$

已经是有序的。我们将 $arr[i]$ 不断向左移动，直到它到达正确的位置。

时间复杂度

在最坏情况下，插入排序的时间复杂度为

$$O(n^2).$$

---

2 用递归求最大值

代码

```java
public class Code08_getMax {
    public static int getMax(int[] arr) {
        return process(arr, 0, arr.length - 1);
    }

    // 在 arr[L..R] 中求最大值
    public static int process(int[] arr, int L, int R) {
        if (L == R) {
            return arr[L];
        }
        int mid = L + ((R - L) >> 1);
        int leftMax = process(arr, L, mid);
        int rightMax = process(arr, mid + 1, R);
        return Math.max(leftMax, rightMax);
    }
}
```

思想

这个方法使用了递归和分治。
为了求 $arr[L \ldots R]$ 中的最大值：

- 如果只有一个元素，就直接返回它；
- 否则，把区间分成左右两半；
- 递归求左半部分的最大值；
- 递归求右半部分的最大值；
- 返回两者中较大的那个。

例子

假设

$$arr = [3, 8, 2, 7, 5, 1].$$

那么

$$getMax(arr) = process(arr, 0, 5).$$

递归过程可以画成下面这棵树：

```
process(arr, 0, 5)
|-- process(arr, 0, 2)
|   |-- process(arr, 0, 1)
|   |   |-- process(arr, 0, 0) = 3
|   |   `-- process(arr, 1, 1) = 8
|   |   max(3, 8) = 8
|   `-- process(arr, 2, 2) = 2
|   max(8, 2) = 8
`-- process(arr, 3, 5)
    |-- process(arr, 3, 4)
    |   |-- process(arr, 3, 3) = 7
    |   `-- process(arr, 4, 4) = 5
    |   max(7, 5) = 7
    `-- process(arr, 5, 5) = 1
    max(7, 1) = 7
```

最终结果：

```
max(8, 7) = 8
```

所以最后答案是

$$8.$$

计算机是怎么执行它的

计算机通过调用栈来实现递归。

每一次函数调用都会保存：

- 当前参数 L 和 R；
- 局部变量，例如 `mid`、`leftMax` 和 `rightMax`；
- 返回的位置。

计算会不断向下深入，直到到达基本情况 $L = R$，然后再一层一层向上返回。

核心思想

整个区间的最大值等于下面两者中的较大者：

$$\max\Big(\max(arr[L\ldots mid]),\ \max(arr[mid+1\ldots R])\Big).$$

时间复杂度

每个元素都只会在递归过程中参与一次，因此总时间复杂度为

$$O(n).$$

递归深度大约是

$$O(\log n).$$

---

3 归并排序中的 merge 函数

代码

```
public static void merge(int[] arr, int L, int M, int R) {
    int[] help = new int[R - L + 1];
    int i = 0;
    int p1 = L;
    int p2 = M + 1;
    while (p1 <= M && p2 <= R) {
        help[i++] = arr[p1] <= arr[p2] ? arr[p1++] : arr[p2++];
    }
    while (p1 <= M) {
        help[i++] = arr[p1++];
    }
    while (p2 <= R) {
        help[i++] = arr[p2++];
    }
    for (i = 0; i < help.length; i++) {
        arr[L + i] = help[i];
    }
}
```

思想

函数 `merge(arr, L, M, R)` 假设：

$$arr[L \ldots M] \quad \text{和} \quad arr[M+1 \ldots R]$$

都已经分别排好序。

它的目标是把它们合并成一个有序区间

$$arr[L \ldots R].$$

例子

假设

$$arr = [2, 5, 7, 1, 3, 6],$$

其中

$$L = 0, \quad M = 2, \quad R = 5.$$

那么两段有序部分分别是

$$[2, 5, 7] \quad \text{和} \quad [1, 3, 6].$$

归并过程如下：

$$\begin{aligned}
&\text{比较}2 \text{ 和}1 \to 1,\\
&\text{比较}2 \text{ 和}3 \to 2,\\
&\text{比较}5 \text{ 和}3 \to 3,\\
&\text{比较}5 \text{ 和}6 \to 5,\\
&\text{比较}7 \text{ 和}6 \to 6,\\
&\text{复制剩下的元素}7.
\end{aligned}$$

所以最终归并结果是

$$[1, 2, 3, 5, 6, 7].$$

核心思想

用两个指针分别扫描左右两个有序部分，每次取较小的那个元素放入辅助数组中。

时间复杂度

区间 $[L \ldots R]$ 中的每个元素都只处理一次，所以时间复杂度为

$$O(R - L + 1).$$

4 用归并排序求小和问题

下面这个算法用于计算一个数组区间的**小和**。对于每个元素，它的贡献等于它左边所有比它小的元素之和。等价地，也可以理解为：对于每个元素 $arr[i]$，统计它右边有多少个元素比它大，然后把 $arr[i]$ 加上那么多次。

代码

```
public static int process2(int[] arr, int L, int R) {
    if (L == R) {
        return 0;
    }
    int mid = L + ((R - L) >> 1);
    return process2(arr, L, mid)
         + process2(arr, mid + 1, R)
         + merge(arr, L, mid, R);
}

public static int merge(int[] arr, int L, int M, int R) {
    int[] help = new int[R - L + 1];
    int i = 0;
    int p1 = L;
    int p2 = M + 1;
    int res = 0;
    while (p1 <= M && p2 <= R) {
        res += arr[p1] < arr[p2] ? (R - p2 + 1) * arr[p1] : 0;
        help[i++] = arr[p1] <= arr[p2] ? arr[p1++] : arr[p2++];
    }
    while (p1 <= M) {
        help[i++] = arr[p1++];
    }
```

```
    while (p2 <= R) {
        help[i++] = arr[p2++];
    }
    for (i = 0; i < help.length; i++) {
        arr[L + i] = help[i];
    }
    return res;
}
```

主要思想

这个算法沿用了归并排序的框架。对于区间 $arr[L \dots R]$，设

$$mid = L + \left\lfloor \frac{R-L}{2} \right\rfloor.$$

那么区间 $[L \dots R]$ 上的小和可以分解为

$$\text{smallSum}(L, R) = \text{smallSum}(L, mid) + \text{smallSum}(mid+1, R) + \text{crossSum}(L, mid, R).$$

第一部分是完全落在左半边内部的小和，第二部分是完全落在右半边内部的小和，第三部分是左边元素在左半区、右边元素在右半区时产生的跨区间贡献。

因此，`process2` 递归地计算左半部分的小和与右半部分的小和，而 `merge` 在归并的同时计算跨区间贡献，并把当前区间排好序，供更高层递归使用。

为什么 merge 步骤有效

在执行 `merge(arr, L, M, R)` 时，两部分

$$arr[L \dots M] \quad \text{和} \quad arr[M+1 \dots R]$$

都已经排好序。

假设我们当前比较 `arr[p1]` 和 `arr[p2]`。如果

$$arr[p1] < arr[p2],$$

由于右半部分是有序的，那么

$$arr[p2], arr[p2+1], \dots, arr[R]$$

这些元素也都大于 arr[p1]。因此，arr[p1] 对小和的贡献是

$$(R - p2 + 1) \cdot arr[p1].$$

这正是下面这行代码的含义：

```
res += arr[p1] < arr[p2] ? (R - p2 + 1) * arr[p1] : 0;
```

例子：$[1, 3, 4, 2, 5, 0, 6]$

考虑数组

$$arr = [1, 3, 4, 2, 5, 0, 6].$$

我们计算

$$\texttt{process2(arr, 0, 6)}.$$

首先，

$$\texttt{process2(0,6)} = \texttt{process2(0,3)} + \texttt{process2(4,6)} + \texttt{merge(0,3,6)}.$$

递归分解过程如下：

```
process2(0,6)
|
|-- process2(0,3)
|   |
|   |-- process2(0,1)
|   |   |
|   |   |-- process2(0,0) = 0
|   |   |-- process2(1,1) = 0
|   |   `-- merge(0,0,1)
|   |
|   |-- process2(2,3)
|   |   |
|   |   |-- process2(2,2) = 0
|   |   |-- process2(3,3) = 0
|   |   `-- merge(2,2,3)
|   |
|   `-- merge(0,1,3)
```

```
|
|-- process2(4,6)
|   |
|   |-- process2(4,5)
|   |   |
|   |   |-- process2(4,4) = 0
|   |   |-- process2(5,5) = 0
|   |   `-- merge(4,4,5)
|   |
|   |-- process2(6,6) = 0
|   |
|   `-- merge(4,5,6)
|
`-- merge(0,3,6)
```

每个只含一个元素的区间都返回 0。下面自底向上计算。

第 1 步：process2(0,1) 这个区间是 $[1,3]$。因为

$$\texttt{process2(0,0)} = 0, \qquad \texttt{process2(1,1)} = 0,$$

所以只需要计算 merge(0,0,1)。比较 1 和 3，有

$$1 < 3,$$

所以贡献为

$$(1 - 1 + 1) \cdot 1 = 1.$$

因此

$$\texttt{merge(0,0,1)} = 1, \qquad \texttt{process2(0,1)} = 1.$$

第 2 步：process2(2,3) 这个区间是 $[4,2]$。因为

$$\texttt{process2(2,2)} = 0, \qquad \texttt{process2(3,3)} = 0,$$

所以计算 merge(2,2,3)。比较 4 和 2 时，条件

$$4 < 2$$

不成立，因此贡献是 0。于是

$$\texttt{merge(2,2,3)} = 0, \qquad \texttt{process2(2,3)} = 0.$$

归并后，该区间变成 $[2,4]$。

第 3 步：`process2(0,3)` 现在左半部分是 $[1,3]$，右半部分是 $[2,4]$。我们已经知道

$$\texttt{process2(0,1)} = 1, \qquad \texttt{process2(2,3)} = 0.$$

接下来计算 `merge(0,1,3)`：

- 比较 1 和 2：因为 $1 < 2$，右边还剩两个元素 $[2,4]$，贡献为 $2 \cdot 1 = 2$；
- 比较 3 和 2：因为 $3 < 2$ 不成立，贡献为 0；
- 比较 3 和 4：因为 $3 < 4$，右边还剩一个元素，贡献为 $1 \cdot 3 = 3$。

因此，

$$\texttt{merge(0,1,3)} = 2 + 3 = 5,$$

所以

$$\texttt{process2(0,3)} = 1 + 0 + 5 = 6.$$

归并后，该区间变成

$$[1,2,3,4].$$

第 4 步：`process2(4,5)` 这个区间是 $[5,0]$。由于

$$\texttt{process2(4,4)} = 0, \qquad \texttt{process2(5,5)} = 0,$$

所以计算 `merge(4,4,5)`。因为

$$5 < 0$$

不成立，所以贡献是 0。因此

$$\texttt{merge(4,4,5)} = 0, \qquad \texttt{process2(4,5)} = 0.$$

归并后，该区间变成

$$[0,5].$$

第 5 步：`process2(4,6)` 现在左半部分是 $[0,5]$，右半部分是 $[6]$。因为

$$\texttt{process2(4,5)} = 0, \qquad \texttt{process2(6,6)} = 0,$$

所以计算 `merge(4,5,6)`：

- 比较 0 和 6：贡献 $1 \cdot 0 = 0$；

- 比较 5 和 6：贡献 $1 \cdot 5 = 5$。

于是

$$\texttt{merge(4,5,6)} = 5, \qquad \texttt{process2(4,6)} = 5.$$

归并后，该区间变成

$$[0, 5, 6].$$

第 6 步：`process2(0,6)` 现在左半部分是

$$[1, 2, 3, 4]$$

右半部分是

$$[0, 5, 6].$$

我们已经知道

$$\texttt{process2(0,3)} = 6, \qquad \texttt{process2(4,6)} = 5.$$

现在计算 `merge(0,3,6)`：

- 比较 1 和 0：没有贡献；
- 比较 1 和 5：贡献 $2 \cdot 1 = 2$；
- 比较 2 和 5：贡献 $2 \cdot 2 = 4$；
- 比较 3 和 5：贡献 $2 \cdot 3 = 6$；
- 比较 4 和 5：贡献 $2 \cdot 4 = 8$。

所以

$$\texttt{merge(0,3,6)} = 2 + 4 + 6 + 8 = 20.$$

最终，

$$\texttt{process2(0,6)} = \texttt{process2(0,3)} + \texttt{process2(4,6)} + \texttt{merge(0,3,6)} = 6 + 5 + 20 = 31.$$

最终答案

因此，

$$[1, 3, 4, 2, 5, 0, 6]$$

的小和是

$$31.$$

总结

这个算法先把数组不断拆分成更小的区间，直到每个区间只剩一个元素。然后自底向上计算结果：

$$\text{smallSum}(L, R) = \text{smallSum}(L, mid) + \text{smallSum}(mid + 1, R) + \text{crossSum}(L, mid, R).$$

`merge` 这一步非常关键，因为它在完成归并排序的同时，也高效地计算了跨区间贡献。因此总时间复杂度为

$$O(n \log n).$$

5 使用荷兰国旗划分的快速排序

下面的代码实现了带随机主元和三路划分的快速排序。它的思想是把数组分成三部分：

$$< \text{pivot}, \qquad = \text{pivot}, \qquad > \text{pivot},$$

然后只递归排序左边和右边两部分。

代码

```
public static void quickSort(int[] arr, int L, int R) {
    if (L < R) {
        swap(arr, L + (int) (Math.random() * (R - L + 1)), R);
        int[] p = partition(arr, L, R);
        quickSort(arr, L, p[0] - 1);   // < pivot 区
        quickSort(arr, p[1] + 1, R);   // > pivot 区
    }
}

public static int[] partition(int[] arr, int L, int R) {
    int less = L - 1;
    int more = R;
    while (L < more) {
        if (arr[L] < arr[R]) {
            swap(arr, ++less, L++);
        } else if (arr[L] > arr[R]) {
            swap(arr, --more, L);
```

```
        } else {
            L++;
        }
    }
    swap(arr, more, R);
    return new int[] {less + 1, more};
}
```

主要思想

函数 `quickSort(arr, L, R)` 会先从区间 $[L, R]$ 中随机选出一个元素，并把它与 `arr[R]` 交换。因此，主元 pivot 被放在位置 R。

接着，`partition(arr, L, R)` 会把当前区间重排成三部分：

$$[L \ldots less] < \text{pivot},$$

$$[less + 1 \ldots more] = \text{pivot},$$

$$[more + 1 \ldots R] > \text{pivot}.$$

之后，算法只递归排序下面两个子区间：

$$[L \ldots p[0] - 1] \quad \text{和} \quad [p[1] + 1 \ldots R].$$

中间这一段全都是 pivot，不需要再递归。

变量含义

在 `partition` 函数中，维护了三个区域：

- `less`：表示小于 pivot 区域的右边界；
- `more`：表示大于 pivot 区域的左边界；
- `L`：当前扫描到的位置。

在循环执行的任意时刻，数组结构如下：

```
[L0 ... less]        < pivot
[less+1 ... L-1]     = pivot
[L ... more-1]       未处理
[more ... R-1]       > pivot
[R]                  pivot
```

这里的 $L0$ 表示当前区间原始的左边界。

为什么三种情况都成立

在循环中，我们把 `arr[L]` 和主元 `arr[R]` 进行比较。

情况 1：`arr[L] < pivot` 那么当前元素应属于左边区域。我们把 "小于区" 扩大一格，并把当前元素交换进去：

```
swap(arr, ++less, L++);
```

做完之后，当前位置上的元素已经处理完毕，因此可以安全地把 L 向右移动。

情况 2：`arr[L] > pivot` 那么当前元素应属于右边区域。我们通过左移 `more` 来扩大 "大于区"，并把当前元素交换进去：

```
swap(arr, --more, L);
```

这里**不能**增加 L，因为交换到位置 L 的新元素还没有检查过。

情况 3：`arr[L] == pivot` 那么当前元素本来就属于中间区域，所以直接执行

```
L++;
```

为什么最后还要交换一次

循环停止时，满足 $L = more$。这时数组结构是：

```
[L0 ... less]        < pivot
[less+1 ... more-1]  = pivot
[more ... R-1]       > pivot
[R]                  pivot
```

此时真正的 pivot 还在位置 R。所以我们要把它与 `arr[more]` 交换：

```
swap(arr, more, R);
```

交换之后，数组变成：

```
[L0 ... less]        < pivot
[less+1 ... more]    = pivot
[more+1 ... R]       > pivot
```

因此，等于区正好就是

$$[less + 1 \ldots more].$$

例子

考虑数组

$$[5, 3, 7, 2, 3, 8, 1, 6, 3, 4].$$

假设随机选到的主元是 3，并已经把它交换到了最后，那么参与划分的区间变成

$$[5, 3, 7, 2, 4, 8, 1, 6, 3, 3],$$

其中最后一个元素就是 pivot。

初始时，

$$less = -1, \qquad more = 9, \qquad L = 0.$$

下面一步一步处理。

第 1 步：检查 5　因为

$$5 > 3,$$

所以把它交换到“大于区”，执行

`swap(arr, --more, L).`

于是 `more` 变成 8，交换位置 8 和 0 后，数组变成

$$[3, 3, 7, 2, 4, 8, 1, 6, 5, 3].$$

此时 L 不变，仍然是 0。

第 2 步：检查 3　现在

$$3 = 3,$$

所以它属于中间区，只需令

$$L = 1.$$

第 3 步：检查 3　同理，

$$3 = 3,$$

所以令

$$L = 2.$$

第 4 步：检查 7　因为

$$7 > 3,$$

所以把它移到“大于区”。此时 `more` 变成 7，交换位置 7 和 2 后，数组变成

$$[3, 3, 6, 2, 4, 8, 1, 7, 5, 3].$$

此时 L 不变，仍为 2。

第 5 步：检查 6　因为

$$6 > 3,$$

再次把它移到“大于区”。此时 `more` 变成 6，交换位置 6 和 2 后，数组变成

$$[3, 3, 1, 2, 4, 8, 6, 7, 5, 3].$$

此时 L 仍为 2。

第 6 步：检查 1　因为

$$1 < 3,$$

所以把它移到“小于区”，执行

`swap(arr, ++less, L++)`.

此时 `less` 变成 0，交换位置 0 和 2 后，数组变成

$$[1, 3, 3, 2, 4, 8, 6, 7, 5, 3].$$

然后 L 增加为 3。

第 7 步：检查 2　因为

$$2 < 3,$$

所以把它移到“小于区”。此时 `less` 变成 1，交换位置 1 和 3 后，数组变成

$$[1, 2, 3, 3, 4, 8, 6, 7, 5, 3].$$

然后 $L = 4$。

第 8 步：检查 4　因为

$$4 > 3,$$

所以把它移到“大于区”。此时 `more` 变成 5，交换位置 5 和 4 后，数组变成

$$[1, 2, 3, 3, 8, 4, 6, 7, 5, 3].$$

此时保持 $L = 4$。

第 9 步：检查 8　因为

$$8 > 3,$$

所以继续把它移到“大于区”。此时 `more` 变成 4。位置 4 和 4 交换，数组不变：

$$[1, 2, 3, 3, 8, 4, 6, 7, 5, 3].$$

现在 $L = 4$ 且 $more = 4$，循环结束。

最后交换　最后交换 `arr[more]` 和 `arr[R]`，也就是交换位置 4 和 9。数组变成

$$[1, 2, 3, 3, 3, 4, 6, 7, 5, 8].$$

因此划分后：

- 小于 3 的区域：$[1, 2]$；
- 等于 3 的区域：$[3, 3, 3]$；
- 大于 3 的区域：$[4, 6, 7, 5, 8]$。

所以 `partition` 返回

$$[2, 4],$$

表示等于区的下标范围是从 2 到 4。

快速排序下一步做什么

完成划分后，中间的等于区已经处理好了，因此快速排序只需要递归排序两边：

$$\texttt{quickSort(arr, 0, 1)} \quad \text{和} \quad \texttt{quickSort(arr, 5, 9)}.$$

所以快速排序会不断重复下面三个步骤：

1. 随机选择一个主元；
2. 按照 $<$、$=$、$>$ 三部分进行划分；
3. 递归排序左右两部分。

总结

这个版本的快速排序结合了两个重要思想：

- **随机选择主元**，降低出现最坏情况的概率；
- **三路划分**，能够高效处理重复元素。

划分过程中的关键点是：

$$\texttt{arr[L] < pivot} \Rightarrow \text{放到左边，并且} L++,$$

$$\texttt{arr[L] > pivot} \Rightarrow \text{放到右边，但} L \text{ 不增加},$$

$$\texttt{arr[L] = pivot} \Rightarrow \text{直接} L++.$$

平均情况下，快速排序的时间复杂度是

$$O(n \log n).$$

6 堆排序

代码

```java
public class BinaryTree  {
    public static void heapSort(int[] arr) {
        if (arr == null || arr.length < 2) {
            return;
        }

        for (int i = 0; i < arr.length; i++) { // O(N)
            heapInsert(arr, i); // O(logN)
        }

        int heapSize = arr.length;
        swap(arr, 0, --heapSize);

        while(heapSize > 0) { // O(N)
            heapify(arr, 0, heapSize); // O(logN)
            swap(arr, 0, --heapSize); // O(1)
        }
    }

    // 某个数现在处在index位置，往上继续移动
    public static void heapInsert(int[] arr, int index) {
        while(arr[index] > arr[(index - 1) / 2]) {
            swap(arr, index, (index - 1) / 2);
            index = (index - 1) / 2;
        }
    }

    // 某个数处在index位置，能否向下移动
    public static void heapify(int[] arr, int index, int heapSize) {
        int left = index * 2 + 1; // 左孩子的下标

        while (left < heapSize) { // 在下方还有孩子的时候
            // 两个孩子中，谁的值大，把下标给largest
            int largest = left + 1 < heapSize && arr[left + 1] > arr[
                left] ? left + 1 : left;
            // 父和孩子之间谁的值大，把下标给largest
```

```
            largest = arr[largest] > arr[index] ? largest : index;
            if (largest == index) {
                break;
            }
            swap(arr, largest, index);
            index = largest;
            left = index * 2 + 1;
        }
    }
}
```

思想

堆排序的核心思想是：

- 先把整个数组调整成一个**大根堆**；
- 此时堆顶元素就是当前的最大值；
- 把堆顶元素和堆中最后一个元素交换；
- 缩小堆的范围；
- 再对新的堆顶执行 `heapify`，恢复大根堆；
- 重复这个过程，直到整个数组有序。

这里的大根堆满足：

$$\text{每个父节点的值都大于等于它左右孩子的值。}$$

因此整个堆中的最大值一定在根节点，也就是数组下标 0 的位置。

数组和完全二叉树的对应关系

在数组表示的完全二叉树中：

$$\text{左孩子下标} = 2i + 1, \qquad \text{右孩子下标} = 2i + 2, \qquad \text{父节点下标} = \left\lfloor \frac{i-1}{2} \right\rfloor.$$

因此数组可以自然看成一棵完全二叉树。

两个关键函数

1. heapInsert 函数 heapInsert(arr, index) 表示：

某个新加入堆的元素现在位于 index 位置，检查它是否比父节点大；如果更大，就与父节点交换，并继续向上移动，直到不再违反大根堆性质。

所以它也叫**向上调整**。

2. heapify 函数 heapify(arr, index, heapSize) 表示：

某个元素现在位于 index 位置，检查它是否比孩子小；如果比孩子小，就和较大的那个孩子交换，并继续向下移动，直到恢复大根堆性质。

所以它也叫**向下调整**。

整体过程

堆排序分成两个阶段。

第一阶段：建堆 代码中这部分：

```
for (int i = 0; i < arr.length; i++) {
    heapInsert(arr, i);
}
```

表示从左到右依次把元素插入堆中，并通过 heapInsert 逐步把整个数组变成大根堆。

第二阶段：排序 当数组已经是大根堆后，堆顶一定是当前最大值。于是每次：

- 把堆顶和当前堆的最后一个元素交换；
- 让堆的大小减一；
- 对新的堆顶执行 heapify；
- 继续下一轮。

这样就能不断把最大值放到数组右边，最终完成升序排序。

例子

考虑数组

$$arr = [4, 7, 2, 9, 1, 5, 3, 8].$$

它对应的完全二叉树是：

```
          4
       /     \
      7       2
    /   \   /   \
   9     1 5     3
  /
 8
```

这棵树有四层。

建堆过程

我们从左到右依次执行：

`heapInsert(arr,0)`, `heapInsert(arr,1)`, ..., `heapInsert(arr,7)`.

第 1 步 只有一个元素：

$$[4]$$

它本身就是堆。

第 2 步 插入 7 后，数组变成

$$[4, 7].$$

因为

$$7 > 4,$$

所以与父节点交换，得到

$$[7, 4].$$

第 3 步 插入 2 后：

$$[7, 4, 2].$$

因为

$$2 < 7,$$

所以不需要交换。

第 4 步 插入 9 后：

$$[7, 4, 2, 9].$$

先比较 9 和父节点 4，因为

$$9 > 4,$$

交换后得到

$$[7, 9, 2, 4].$$

再比较 9 和新的父节点 7，因为

$$9 > 7,$$

继续交换，得到

$$[9, 7, 2, 4].$$

第 5 步 插入 1 后：

$$[9, 7, 2, 4, 1].$$

因为

$$1 < 7,$$

所以不需要调整。

第 6 步 插入 5 后:

$$[9, 7, 2, 4, 1, 5].$$

先比较 5 和父节点 2，因为

$$5 > 2,$$

交换后得到

$$[9, 7, 5, 4, 1, 2].$$

再和父节点 9 比较，因为

$$5 < 9,$$

所以停止。

第 7 步 插入 3 后:

$$[9, 7, 5, 4, 1, 2, 3].$$

因为

$$3 < 5,$$

所以不需要调整。

第 8 步 插入 8 后:

$$[9, 7, 5, 4, 1, 2, 3, 8].$$

先比较 8 和父节点 4，因为

$$8 > 4,$$

交换后得到

$$[9, 7, 5, 8, 1, 2, 3, 4].$$

再比较 8 和父节点 7，因为

$$8 > 7,$$

交换后得到

$$[9, 8, 5, 7, 1, 2, 3, 4].$$

再比较 8 和父节点 9，因为

$$8 < 9,$$

所以停止。

于是建堆完成，最终得到的大根堆是：

$$[9, 8, 5, 7, 1, 2, 3, 4].$$

对应的二叉树为：

```
        9
      /     \
     8       5
   /   \   /   \
  7     1 2     3
 /
4
```

排序过程

第 1 轮 当前堆是：

$$[9, 8, 5, 7, 1, 2, 3, 4].$$

交换堆顶和最后一个元素：

$$[4, 8, 5, 7, 1, 2, 3, 9].$$

此时 9 已经排好序，堆大小变成 7。

对下标 0 执行 `heapify`：

- 比较左右孩子 8 和 5，较大的是 8；
- 因为 $8 > 4$，交换；
- 得到 $[8, 4, 5, 7, 1, 2, 3, 9]$；
- 继续向下看 4；
- 比较它的两个孩子 7 和 1，较大的是 7；

- 因为 $7 > 4$，交换；
- 得到 $[8, 7, 5, 4, 1, 2, 3, 9]$。

所以第 1 轮结束后，数组为：

$$[8, 7, 5, 4, 1, 2, 3, 9].$$

第 2 轮 交换堆顶和当前最后一个元素：

$$[3, 7, 5, 4, 1, 2, 8, 9].$$

堆大小变成 6。
执行 `heapify`：

- 比较 7 和 5，较大的是 7；
- 因为 $7 > 3$，交换，得到 $[7, 3, 5, 4, 1, 2, 8, 9]$；
- 再比较 3 的孩子 4 和 1，较大的是 4；
- 因为 $4 > 3$，交换，得到 $[7, 4, 5, 3, 1, 2, 8, 9]$。

第 3 轮 交换堆顶和当前最后一个元素：

$$[2, 4, 5, 3, 1, 7, 8, 9].$$

堆大小变成 5。
执行 `heapify`：

- 比较 4 和 5，较大的是 5；
- 因为 $5 > 2$，交换，得到 $[5, 4, 2, 3, 1, 7, 8, 9]$。

第 4 轮 交换堆顶和当前最后一个元素：

$$[1, 4, 2, 3, 5, 7, 8, 9].$$

堆大小变成 4。
执行 `heapify`：

- 比较 4 和 2，较大的是 4；
- 因为 $4 > 1$，交换，得到 $[4, 1, 2, 3, 5, 7, 8, 9]$；
- 再比较 1 和它的孩子 3，因为 $3 > 1$，再交换；
- 得到 $[4, 3, 2, 1, 5, 7, 8, 9]$。

第 5 轮　交换堆顶和当前最后一个元素：

$$[1, 3, 2, 4, 5, 7, 8, 9].$$

堆大小变成 3。
执行 `heapify`：

- 比较 3 和 2，较大的是 3；
- 因为 $3 > 1$，交换，得到 $[3, 1, 2, 4, 5, 7, 8, 9]$。

第 6 轮　交换堆顶和当前最后一个元素：

$$[2, 1, 3, 4, 5, 7, 8, 9].$$

堆大小变成 2。
执行 `heapify` 后不再需要交换。

第 7 轮　交换最后两个元素：

$$[1, 2, 3, 4, 5, 7, 8, 9].$$

排序完成。

为什么这个算法是对的

每一轮都有两个关键事实：

- 当前堆顶一定是未排序部分中的最大值；
- 把它交换到当前堆的最后一个位置后，这个位置就已经确定了。

因此，第一轮确定最大值，第二轮确定第二大值，依次类推，最后整个数组就会按升序排列。

核心思想

堆排序的本质可以概括为：

- 先把数组组织成一棵“最大值在最上面”的树；
- 然后不断把堆顶最大值取出来，放到数组右边已经排好序的位置。

时间复杂度

这段代码中，建堆方式是逐个调用 `heapInsert`：

$$n \text{ 次}\texttt{heapInsert}, \qquad O(\log n).$$

所以建堆复杂度为

$$O(n \log n).$$

之后排序阶段，每轮一次 `heapify`，总共执行 n 轮，因此复杂度也是

$$O(n \log n).$$

所以整个堆排序的时间复杂度为

$$O(n \log n).$$

空间复杂度为

$$O(1),$$

因为它是原地排序。

总结

堆排序可以分成两步：

1. 建立大根堆；
2. 反复把堆顶最大值交换到数组末尾，并缩小堆范围。

其中：

- heapInsert 负责向上调整；
- heapify 负责向下调整。

平均和最坏情况下，堆排序的时间复杂度都是

$$O(n \log n).$$

7 题目

给定一个整型数组 arr，已知这个数组是**几乎有序**的，也就是说：

如果把数组排好序，那么每个元素移动的距离都不会超过 k。

这里“移动的距离不超过 k”的意思是：

对任意元素而言，它在原数组中的位置与它在最终有序数组中的位置之差的绝对值不超过k.

要求设计一个高效算法，对这个数组进行排序。

8 代码

```
public void sortedArrDistanceLessK(int[] arr, int k) {
    // 默认小根堆
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    int index = 0;

    for (; index < Math.min(arr.length, k); index++) {
        heap.add(arr[index]);
    }

    int i = 0;
    for (; index < arr.length; i++, index++) {
        heap.add(arr[index]);
        arr[i] = heap.poll();
    }
```

```
    while (!heap.isEmpty()) {
        arr[i++] = heap.poll();
    }
}
```

9 思想

这道题的关键条件是：

$$\text{每个元素距离它最终排序后的位置最多不超过}k.$$

因此，当我们要确定排好序后第 i 个位置的数时，这个数不可能来自特别远的地方。它只可能来自当前附近的一小段区间。

所以没有必要每次都在整个剩余数组里找最小值，只需要在一个大小大约为 k 的候选范围内找最小值即可。

为了高效得到这批候选元素中的最小值，代码使用了：

```
PriorityQueue<Integer>
```

在 Java 中，默认就是**小根堆**。

也就是说：

```
heap.poll()
```

每次都会弹出当前堆中最小的元素。

10 为什么可以用小根堆

因为数组中每个元素离最终位置最多只差 k，所以：

- 最终排在第 0 个位置的元素，一定在原数组前面不远的地方；
- 最终排在第 1 个位置的元素，也一定在接下来不远的范围里；
- 以此类推。

因此，我们可以始终维护一个“小窗口”中的候选元素，并用小根堆保存它们。
这样每一步只需要：

- 把一个新元素加入堆；
- 从堆中弹出最小值；
- 把这个最小值写回数组当前位置。

11 代码分三部分

第一部分：先把前面一批元素放进堆

```
for (; index < Math.min(arr.length, k); index++) {
    heap.add(arr[index]);
}
```

这一段的作用是先建立初始的小根堆。
如果数组长度小于 k，那么就把整个数组都放进去；否则就先放前 k 个元素进去。

第二部分：一边加入新元素，一边弹出最小值

```
int i = 0;
for (; index < arr.length; i++, index++) {
    heap.add(arr[index]);
    arr[i] = heap.poll();
}
```

这一段是核心。
每一轮都做两件事：

1. 把当前新的元素加入堆；
2. 弹出堆中最小的元素，放到数组当前位置 i。

因为堆中保存的正是“当前位置可能出现的候选元素”，所以弹出的最小值就是当前最应该放在前面的元素。

第三部分：把堆中剩下的元素依次弹出

```
while (!heap.isEmpty()) {
    arr[i++] = heap.poll();
}
```

当前面的循环结束时，所有数组元素都已经加入过堆了，但堆里可能还剩下一些元素没有写回数组。

由于小根堆每次弹出的都是最小值，因此直接依次弹出即可。

12 一个较大的例子

考虑下面这个数组：

$$arr = [4, 2, 1, 7, 5, 3, 8, 6, 9, 10]$$

设

$$k = 3.$$

这个数组是几乎有序的，因为每个元素离最终位置都不远。它排序后的结果应为：

$$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10].$$

初始入堆

根据代码，先把前 k 个元素放进堆，也就是：

$$4,\ 2,\ 1$$

放入小根堆后，可以画成下面这棵树（逻辑结构）：

```
  1
 / \
4   2
```

这里根节点 1 最小，所以它会最先被弹出。

第 1 轮

此时 $index = 3$，把

$$arr[3] = 7$$

加入堆中。
堆中的元素变成：

$$\{1, 2, 4, 7\}$$

对应的小根堆结构可以看成：

```
      1
    /   \
  4       2
 /
7
```

然后执行：

$$arr[0] = heap.poll()$$

弹出最小值 1，于是：

$$arr[0] = 1.$$

第 2 轮

把

$$arr[4] = 5$$

加入堆中。此时堆里大致是：

$$\{2, 4, 7, 5\}$$

小根堆可以表示为：

```
      2
    /   \
   4     7
  /
 5
```

然后弹出最小值 2，写入：

$$arr[1] = 2.$$

第 3 轮

把

$$arr[5] = 3$$

加入堆中。此时堆里的元素是：

$$\{3, 4, 7, 5\}$$

堆结构可以表示为：

```
      3
    /   \
   4     7
  /
 5
```

然后弹出最小值 3，写入：

$$arr[2] = 3.$$

第 4 轮

把

$$arr[6] = 8$$

加入堆中。此时堆里元素是：

$$\{4, 5, 7, 8\}$$

对应的小根堆可以看成：

```
      4
    /   \
   5     7
  /
 8
```

然后弹出最小值 4，写入：

$$arr[3] = 4.$$

第 5 轮

把

$$arr[7] = 6$$

加入堆中。此时堆里元素是：

$$\{5, 6, 7, 8\}$$

对应的小根堆可表示为：

```
      5
    /   \
   6     7
  /
 8
```

然后弹出最小值 5，写入：

$$arr[4] = 5.$$

第 6 轮

把

$$arr[8] = 9$$

加入堆中，堆里元素为：

$$\{6, 8, 7, 9\}$$

可以表示为：

```
      6
    /   \
   8     7
  /
 9
```

然后弹出最小值 6，写入：

$$arr[5] = 6.$$

第 7 轮

把

$$arr[9] = 10$$

加入堆中，堆里元素为：

$$\{7, 8, 9, 10\}$$

对应的小根堆可表示为：

```
      7
    /   \
   8     9
  /
 10
```

然后弹出最小值 7，写入：

$$arr[6] = 7.$$

最后清空堆

此时原数组中的所有元素都已经加入过堆，但堆中还剩：

$$\{8, 9, 10\}$$

继续依次弹出：

$$arr[7] = 8, \qquad arr[8] = 9, \qquad arr[9] = 10.$$

最终数组变成：

$$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10].$$

13 为什么这个算法是对的

关键原因在于：

$$\text{每个元素离最终位置不超过}k.$$

所以当我们要确定当前应放在位置 i 的数时，它不可能来自太远的位置，只会在当前附近的候选集合里。

而小根堆正好可以帮助我们快速找到这批候选元素中的最小值。

因此，每次把堆中的最小值放到数组当前位置，就是正确的。

14 时间复杂度

设数组长度为 n。

堆的大小始终不超过 k 量级，因此：

- 每次 `heap.add()` 的复杂度为 $O(\log k)$；
- 每次 `heap.poll()` 的复杂度也为 $O(\log k)$。

整个过程中，数组中的每个元素大致都会：

- 入堆一次；
- 出堆一次。

所以总时间复杂度为：

$$O(n \log k).$$

15 空间复杂度

由于堆中最多只会存放大约 k 个元素，因此额外空间复杂度为：

$$O(k).$$

16 总结

这道题利用了“每个元素离最终位置不超过 k”这个特殊条件，把原本可能需要 $O(n \log n)$ 的排序过程优化成了：

$$O(n \log k).$$

其核心做法是：

- 用一个小根堆维护当前位置可能出现的候选元素；
- 每次加入一个新元素；
- 每次弹出堆中最小元素写回数组。